VHDL Assignment-3

Latch & Flip-Flop

과목 Digital Logic Circuit Design

학번 2023092479

이름 김우준

제출일 2026. 11. 11

1. 개요

이번 과제에서는 RS Latch, D Latch, D Flip-Flop, JK Flip-Flop, 비동기 Set/Clear가 있는 D Flip-Flop 다섯 가지를 만들었다. 각 문제마다 Logisim에서 게이트만 써서 회로를 그리고(래치나 플립플롭 부품은 쓰지 않음), 같은 동작을 하는 VHDL 코드를 작성했다.

사용한 프로그램은 다음과 같다.

- Logisim-evolution 5.0.0 : 게이트 회로 작성 및 동작 확인
- Questa Altera Starter FPGA Edition 2025.2 : VHDL 컴파일, 시뮬레이션, 회로도(Schematic 창)

VHDL은 문제마다 테스트벤치를 만들어 Questa에서 시뮬레이션했고, 테스트벤치 안에 ASSERT를 넣어 출력이 예상과 다르면 에러가 나오게 했다. 회로도는 Questa에서 `vsim -debugdb`로 시뮬레이션을 연 뒤 Schematic 창에서 띄워 캡처했다. 회로도 위의 숫자는 시뮬레이션이 끝난 시점의 값이다. Logisim 회로는 테스트벤치와 같은 순서로 입력 핀을 눌러 가며 Q, NQ를 확인했다. Logisim 그림에서 밝은 초록색은 1, 진한 초록색은 0이다.

문제	회로	Logisim 구성
1	RS Latch	NOR 2개
2	D Latch	NAND 5개
3	D Flip-Flop	D Latch 2개 + 인버터 1개 (NAND 10, NOT 1)
4	JK Flip-Flop	NAND 15개
5	D Flip-Flop + Set/Clear	NAND 11개 (3입력 8개), NOT 2개

2. 문제 1 - RS Latch

NOR 게이트 두 개의 출력을 서로의 입력으로 연결했다. Q = NOR(R, NQ), NQ = NOR(S, Q)이다. S=1이면 Q=1, R=1이면 Q=0이 되고, 둘 다 0이면 이전 값을 유지한다. S=R=1은 금지 입력으로 Q와 NQ가 둘 다 0이 된다.

S	R	Q	NQ	동작
0	0	Q	NQ	유지
1	0	1	0	Set
0	1	0	1	Reset
1	1	0	0	금지

2.1 Logisim 회로

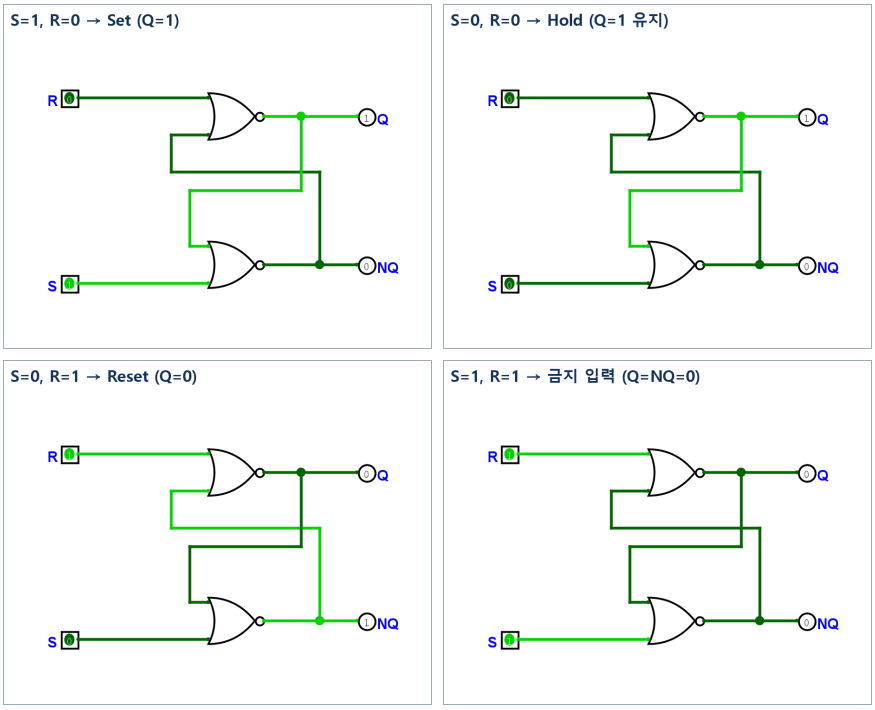


그림 1. RS Latch (Set / 유지 / Reset / 금지 입력)

2.2 VHDL 코드

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- RS latch (NOR)
ENTITY rs_latch IS
  PORT (
    s  : IN  std_logic;   -- Set input
    r  : IN  std_logic;   -- Reset input
    q  : OUT std_logic;   -- Output
    nq : OUT std_logic    -- Complement output
  );
END rs_latch;

ARCHITECTURE bhv OF rs_latch IS
  SIGNAL q_internal  : std_logic := '0';
  SIGNAL nq_internal : std_logic := '1';
BEGIN
  q_internal  <= r NOR nq_internal;
  nq_internal <= s NOR q_internal;

  q  <= q_internal;
  nq <= nq_internal;
END bhv;
```

2.3 회로도 (Questa Schematic)

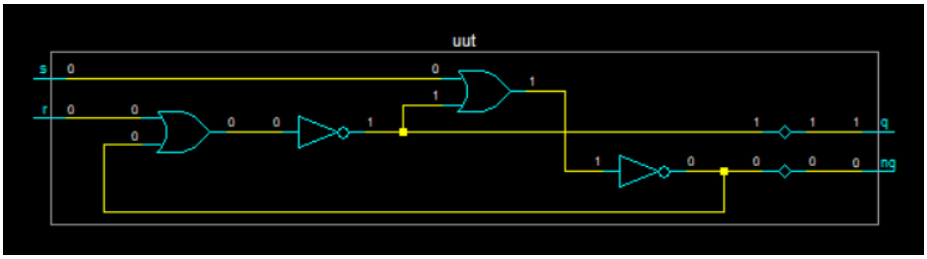


그림 2. rs_latch 회로도. NOR 두 개(OR + NOT)가 서로 되먹임되어 있다

2.4 시뮬레이션 결과

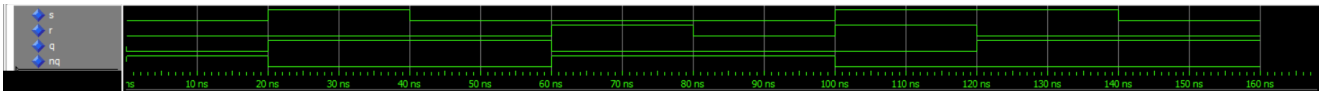


그림 3. tb_rs_latch 파형

시간 (ns)	0-20	20-40	40-60	60-80	80-100	100-120	120-140	140-160
S R	0 0	1 0	0 0	0 1	0 0	1 1	1 0	0 0
Q NQ	0 1	1 0	1 0	0 1	0 1	0 0	1 0	1 0

진리표대로 동작했고 ASSERT 에러는 없었다. 금지 입력 상태에서 S와 R을 동시에 0으로 내리면 두 게이트가 번갈아 바뀌면서 발진할 수 있어서, 테스트에서는 R만 먼저 내려 Set 상태로 빠져나오게 했다(120 ns).

3. 문제 2 - D Latch

강의자료의 D Latch 회로를 NAND만으로 만들었다. 앞쪽 NAND 2개와 D를 반전하는 NAND 1개(두 입력을 묶어 NOT으로 사용), 뒤쪽 NAND SR 래치 2개로 총 5개이다. EN=1이면 Q가 D를 따라가고, EN=0이면 이전 값을 유지한다.

EN	D	Q
0	X	유지
1	0	0
1	1	1

3.1 Logisim 회로

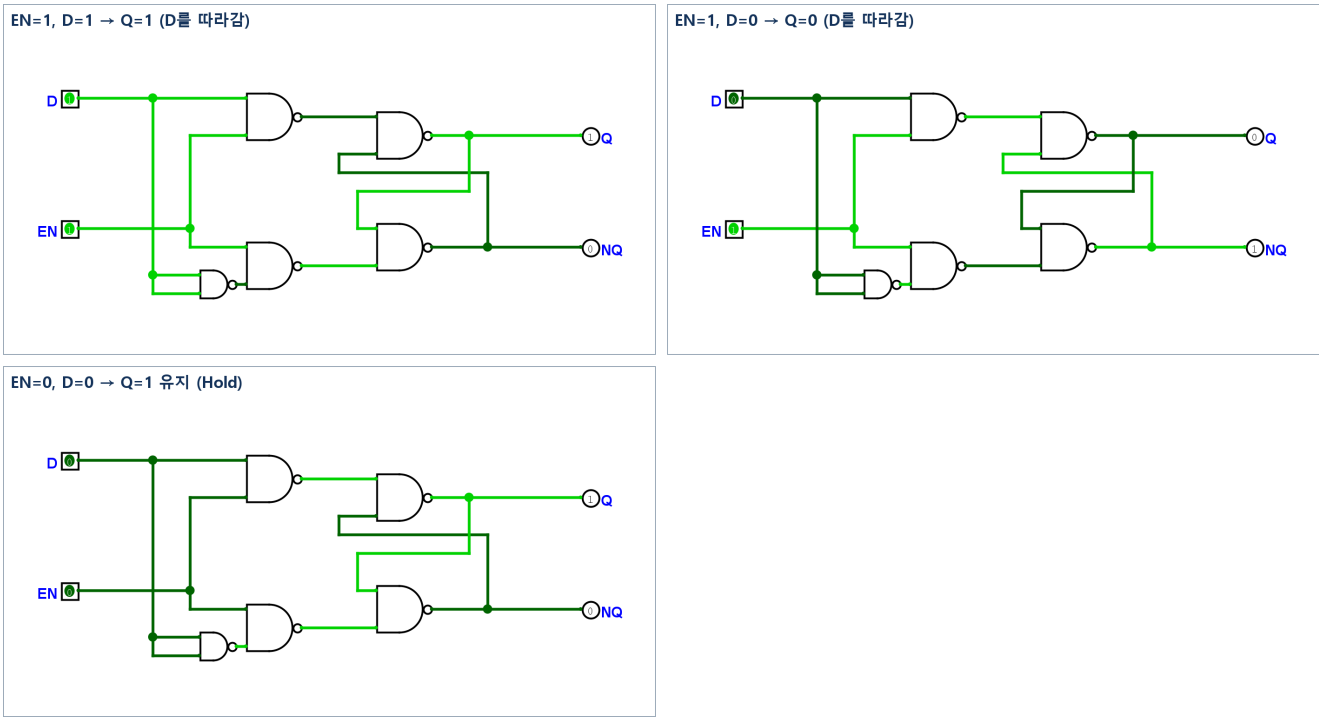


그림 4. D Latch (EN=1일 때 Q가 D를 따라가고, EN=0일 때 1을 유지)

3.2 VHDL 코드

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- D latch with enable
ENTITY d_latch IS
  PORT (
    en : IN  std_logic;   -- Enable signal
    d  : IN  std_logic;   -- Data input
    q  : OUT std_logic;   -- Output
    nq : OUT std_logic    -- Complement output
  );
END d_latch;

ARCHITECTURE bhv OF d_latch IS
  SIGNAL q_internal : std_logic := '0';
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(en, d)
  BEGIN
    IF (en = '1') THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

3.3 회로도 (Questa Schematic)

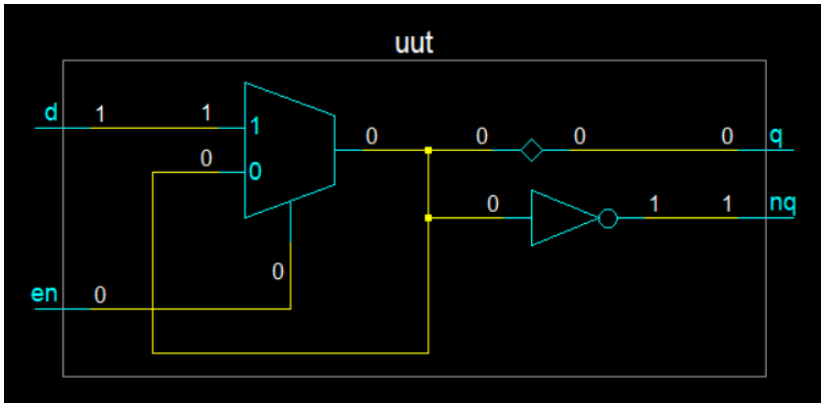


그림 5. d_latch 회로도. en이 1이면 d, 0이면 자기 출력을 선택하는 래치로 나타난다

3.4 시뮬레이션 결과

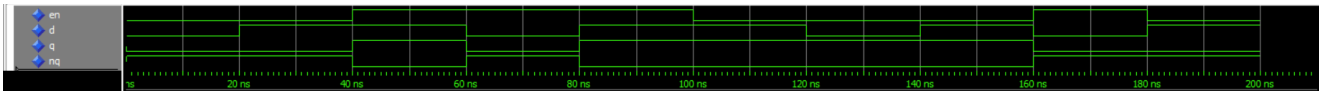


그림 6. tb_d_latch 파형

- 40~100 ns (en=1) : d가 1 → 0 → 1로 바뀌는 대로 q도 바뀐다.
- 100~160 ns (en=0) : d가 바뀌어도 q는 1을 유지한다.
- 160~180 ns에 en=1, d=0이 되어 q=0이 되고, 그 뒤 en=0에서는 d=1이 되어도 q=0을 유지한다.

ASSERT 에러는 없었다.

4. 문제 3 - D Flip-Flop

문제 2의 D Latch 두 개를 Master-Slave로 연결하고 클럭 인버터를 하나 넣었다. Master는 NOT CLK, Slave는 CLK가 enable이다. CLK=0일 때 Master가 D를 받아 두고, CLK가 0에서 1로 바뀌면 Master가 닫히고 Slave가 열려서 그 값이 Q로 나온다. CLK=1인 동안은 Master가 닫혀 있어서 D가 바뀌어도 Q는 그대로이다. 그래서 Q는 상승 에지에서만 바뀐다.

4.1 Logisim 회로

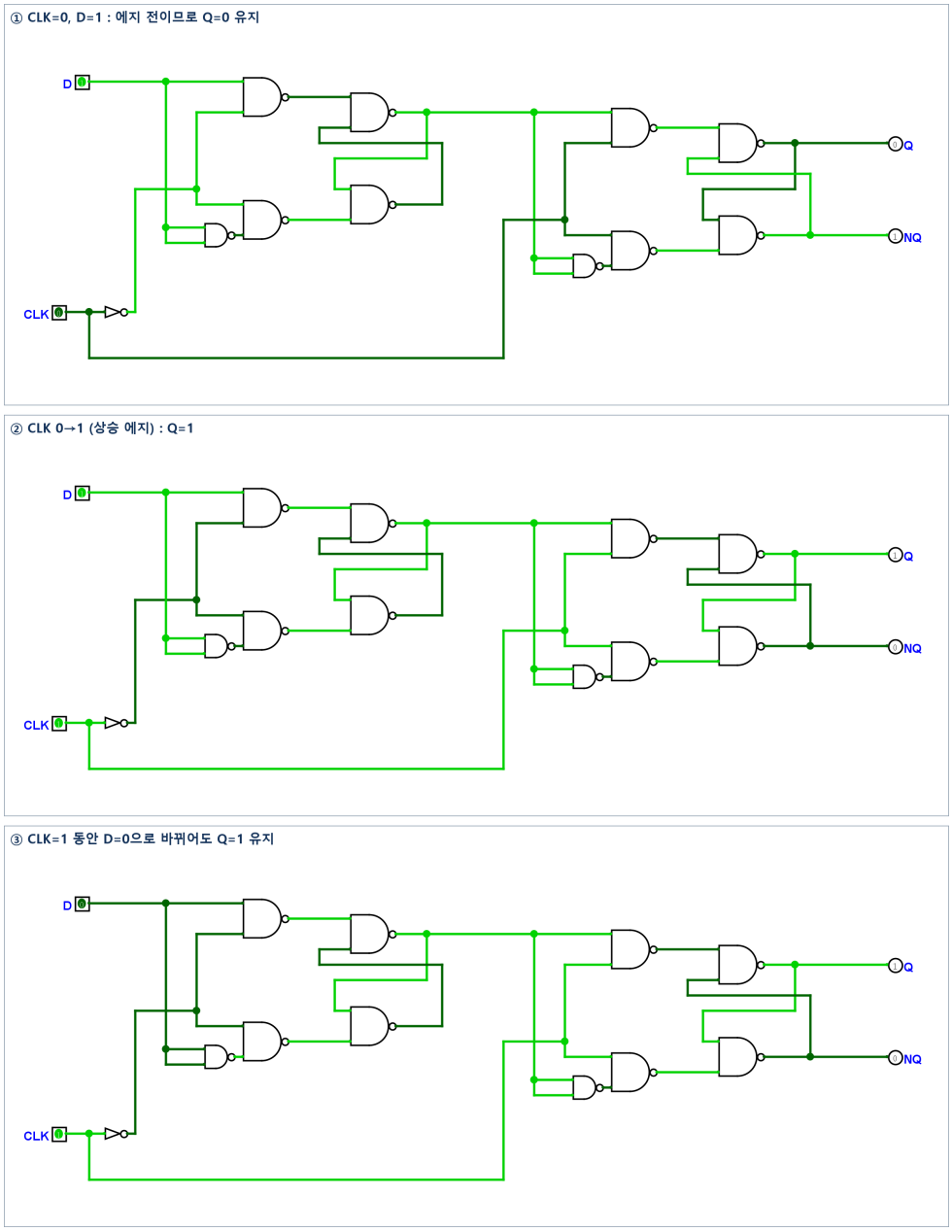


그림 7. Master-Slave D Flip-Flop

4.2 VHDL 코드

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- D flip-flop (rising edge)
ENTITY d_flipflop IS
  PORT (
    clk : IN  std_logic;    -- Clock signal
    d   : IN  std_logic;    -- Data input
    q   : OUT std_logic;    -- Output
    nq  : OUT std_logic     -- Complement output
  );
END d_flipflop;

ARCHITECTURE bhv OF d_flipflop IS
  SIGNAL q_internal : std_logic := '0';
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(clk)
  BEGIN
    IF rising_edge(clk) THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

4.3 회로도 (Questa Schematic)

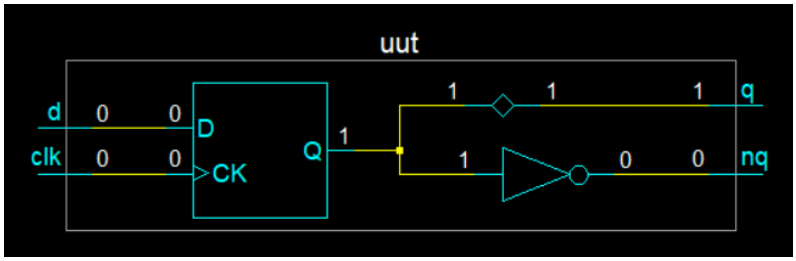


그림 8. d_flipflop 회로도. rising_edge(clk) 부분이 D 플립플롭으로 나타난다

4.4 시뮬레이션 결과

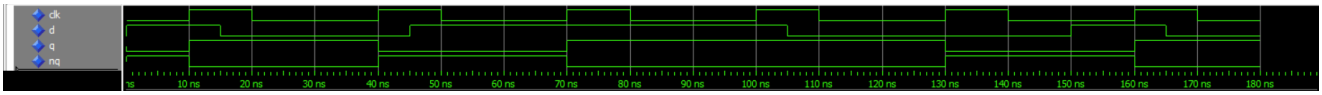


그림 9. tb_d_flipflop 파형

- 상승 에지(10, 40, 70, 100, 130, 160 ns)에서 그때의 d(1, 0, 1, 1, 0, 1)가 q로 들어간다.
- clk가 1인 동안(15 ns 지점) d를 바꿔도, 하강 에지에서도 q는 바뀌지 않는다.

ASSERT 에러는 없었다.

5. 문제 4 - JK Flip-Flop

JK 플립플롭의 특성식 Q(t+1) = J·Q' + K'·Q를 NAND로 만들어서 문제 3과 같은 Master-Slave D Flip-Flop의 D 입력에 넣었다. 입력단은 D = NAND(NAND(J, NQ), NAND(K', Q))이고, 출력 Q와 NQ가 입력단으로 되먹임된다. K'와 클럭 반전도 NAND로 만들어서 전체가 NAND 15개이다.

J	K	Q(t+1)	동작
0	0	Q(t)	유지
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Toggle

처음에는 강의자료 11쪽의 NAND JK 회로를 그대로 그렸는데, CLK=1이고 J=K=1이면 출력이 계속 뒤집히면서 Logisim에서 발진했다(race-around). 과제가 상승 에지 동작이라서 위처럼 Master-Slave 구조 앞에 입력단을 두는 방식으로 바꿨고, 이렇게 하면 에지마다 한 번만 바뀐다.

5.1 Logisim 회로

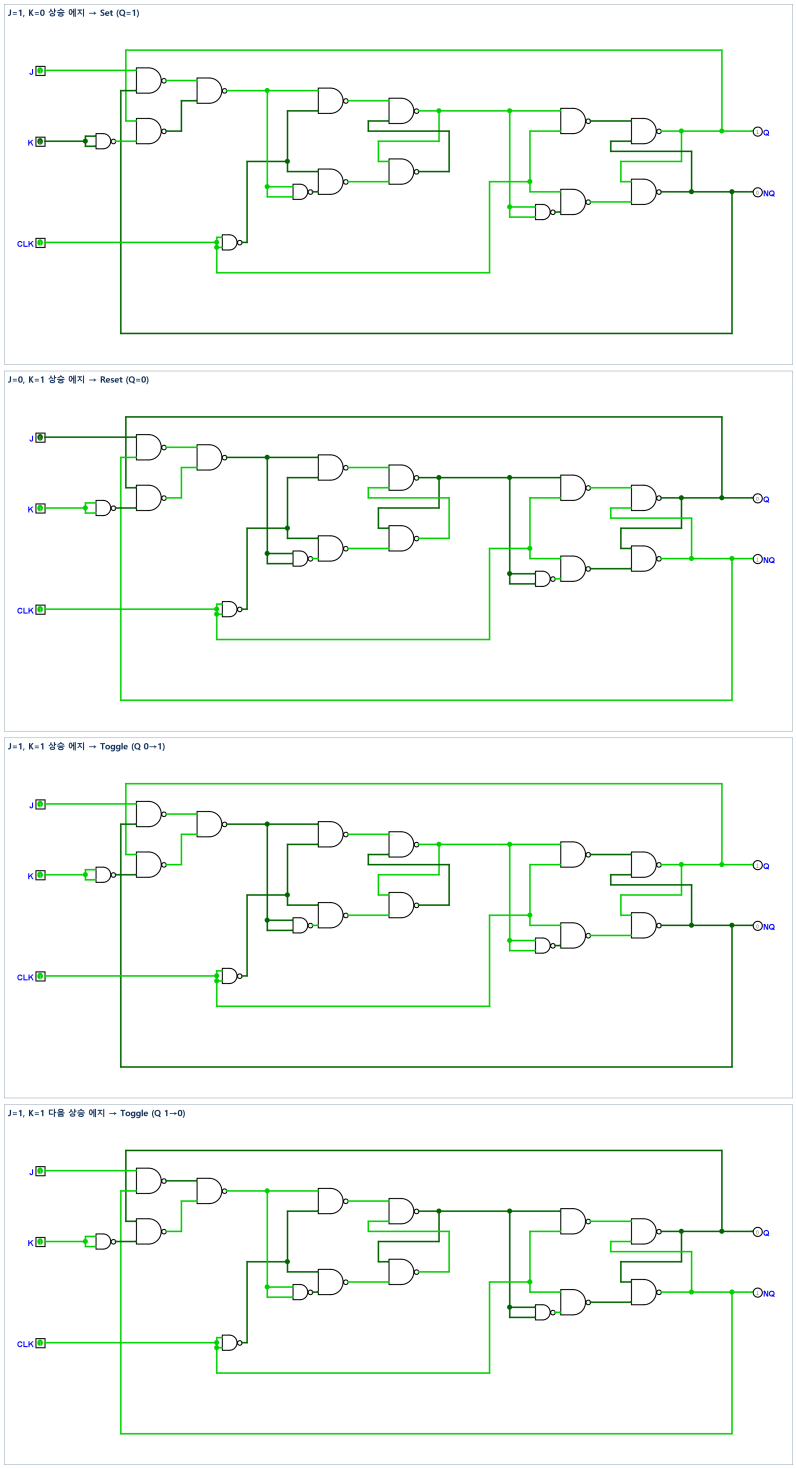


그림 10. JK Flip-Flop (Set / Reset / Toggle 2번)

5.2 VHDL 코드

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- JK flip-flop (rising edge)
-- next state : q+ = j q' + k' q
ENTITY jk_flipflop IS
  PORT (
    clk : IN  std_logic;   -- Clock signal
    j   : IN  std_logic;   -- J input
    k   : IN  std_logic;   -- K input
    q   : OUT std_logic;   -- Output
    nq  : OUT std_logic    -- Complement output
  );
END jk_flipflop;

ARCHITECTURE bhv OF jk_flipflop IS
  SIGNAL q_internal  : std_logic := '0';
  SIGNAL nq_internal : std_logic;
  SIGNAL nk          : std_logic;
  SIGNAL set_term    : std_logic;
  SIGNAL hold_term   : std_logic;
  SIGNAL d_next      : std_logic;
BEGIN
  q  <= q_internal;
  nq <= nq_internal;

  nq_internal <= NOT q_internal;
  nk          <= NOT k;
  set_term    <= j AND nq_internal;
  hold_term   <= nk AND q_internal;
  d_next      <= set_term OR hold_term;

  PROCESS(clk)
  BEGIN
    IF rising_edge(clk) THEN
      q_internal <= d_next;
    END IF;
  END PROCESS;
END bhv;
```

IF / ELSIF로 네 가지 경우를 나눠서 써도 동작은 같지만, Questa 회로도에서 프로세스가 블록 하나로만 보여서 특성식을 게이트 하나씩 나눠 쓰고 클럭 프로세스에서는 d_next를 저장만 하도록 했다.

5.3 회로도 (Questa Schematic)

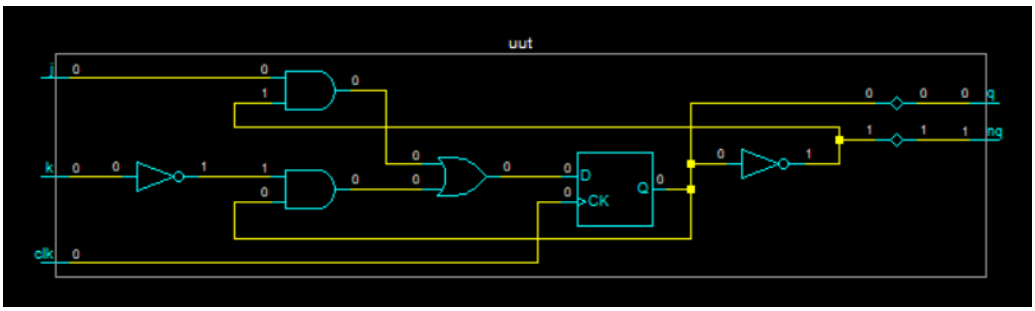


그림 11. jk_flipflop 회로도. NOT(k), AND 2개, OR로 d_next를 만들고 D 플립플롭에 저장한다

5.4 시뮬레이션 결과

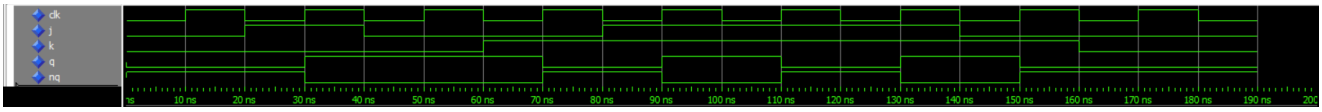


그림 12. tb_jk_flipflop 파형

상승 에지 (ns)	10	30	50	70	90	110	130	150	170
J K	0 0	1 0	0 0	0 1	1 1	1 1	1 1	0 1	0 0
Q	0	1	1	0	1	0	1	0	0

유지, Set, Reset, Toggle 모두 진리표대로 나왔고 ASSERT 에러는 없었다.

6. 문제 5 - 비동기 Set / Clear D Flip-Flop

문제 3의 D Flip-Flop에서 래치 NAND 네 개를 3입력 NAND로 바꾸고, 0일 때 동작하는 nCLR, nPRE 신호를 넣었다. nCLR = NOT clr, nPRE = NAND(set, nCLR)로 만들어서 clr=1이면 nPRE가 1이 되어 Set이 막힌다. 그래서 둘 다 1이면 Clear가 먼저 적용된다. 이 신호들은 래치에 바로 들어가기 때문에 클럭과 상관없이 바로 출력이 바뀐다.

clr	set	clk	Q
1	X	X	0
0	1	X	1
0	0	↑	D
0	0	그 외	유지

6.1 Logisim 회로

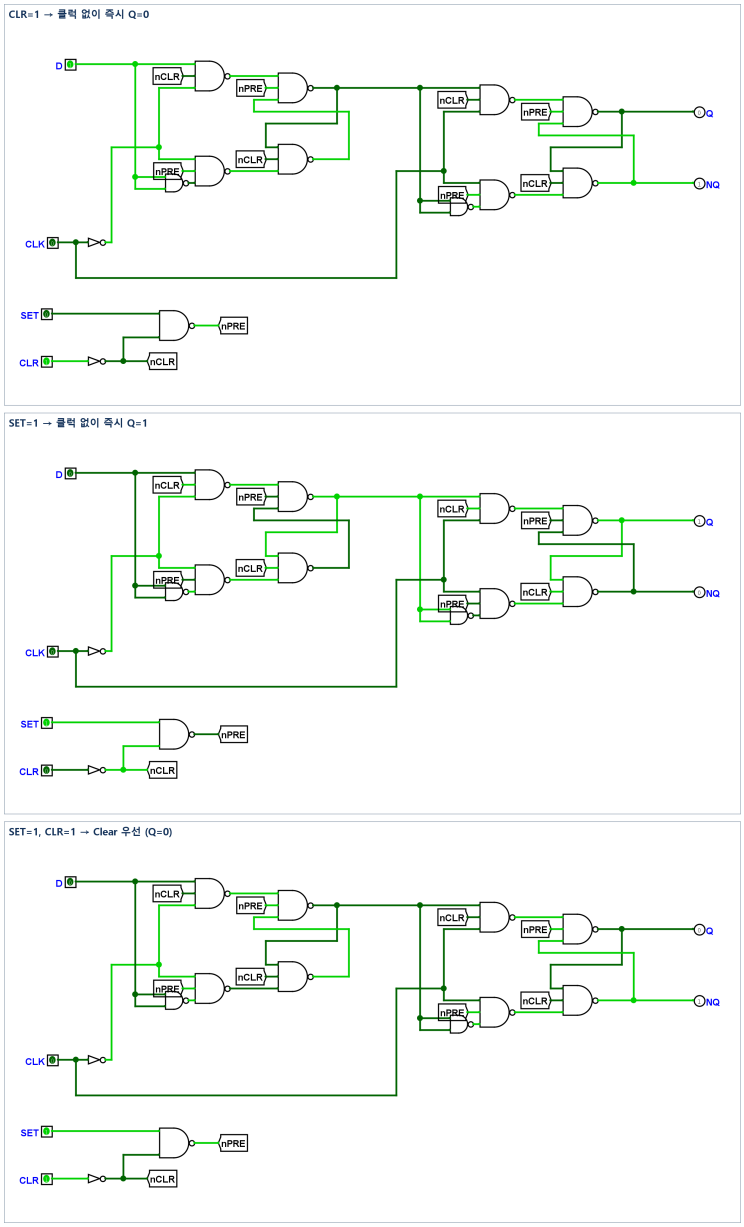


그림 13. CLR / SET이 클럭 없이 바로 적용되고, 둘 다 1이면 Clear

6.2 VHDL 코드

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- D flip-flop with asynchronous set / clear (clr
first)
ENTITY d_flipflop_sc IS
  PORT (
    clk : IN  std_logic;   -- Clock signal
    d   : IN  std_logic;   -- Data input
    clr : IN  std_logic;   -- Asynchronous clear
    set : IN  std_logic;   -- Asynchronous set
    q   : OUT std_logic;   -- Output
    nq  : OUT std_logic    -- Complement output
  );
END d_flipflop_sc;

ARCHITECTURE bhv OF d_flipflop_sc IS
  SIGNAL q_internal : std_logic := '0';
BEGIN
  q  <= q_internal;
  nq <= NOT q_internal;

  PROCESS(clk, clr, set)
  BEGIN
    IF clr = '1' THEN
      q_internal <= '0';
    ELSIF set = '1' THEN
      q_internal <= '1';
    ELSIF rising_edge(clk) THEN
      q_internal <= d;
    END IF;
  END PROCESS;
END bhv;
```

6.3 회로도 (Questa Schematic)

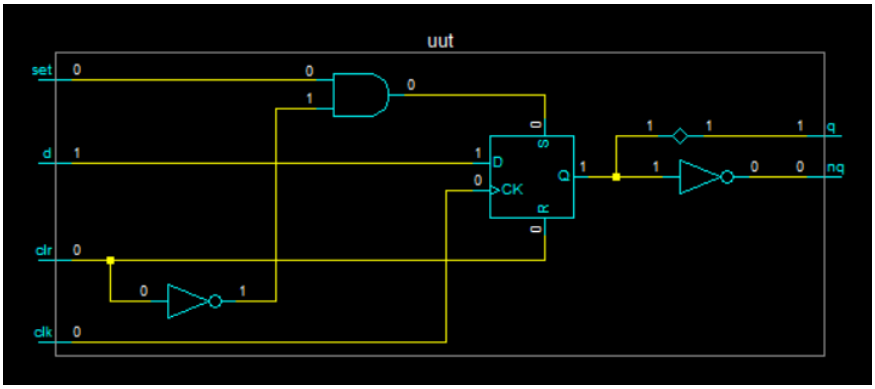


그림 14. d_flipflop_sc 회로도. 플립플롭의 S 단자에 set AND (NOT clr), R 단자에 clr가 연결된다

6.4 시뮬레이션 결과

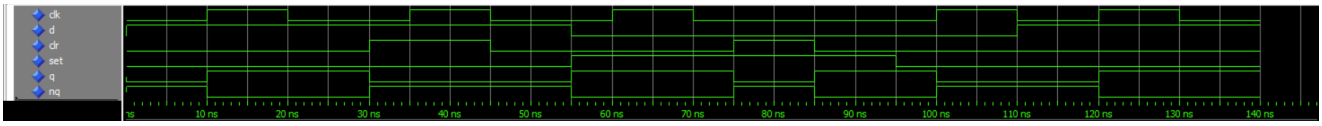


그림 15. tb_d_flipflop_sc 파형

시간 (ns)	10	30	35	55	60	75	85	100	120
입력	clk↑ d=1	clr=1	clk↑	set=1	clk↑ d=0	clr=1 (set=1)	clr=0	clk↑ d=0	clk↑ d=1
Q	1	0	0	1	1	0	1	0	1

clr, set이 들어가는 순간 클럭 없이 q가 바뀌었고, 75 ns처럼 둘 다 1일 때는 q=0이 되었다. ASSERT 에러는 없었다.

7. 결과 및 고찰

다섯 문제 모두 Logisim 회로와 VHDL 시뮬레이션 결과가 같게 나왔다.

- 게이트로 만든 래치는 처음 열었을 때 상태가 정해져 있지 않아서 Logisim에서 Q, NQ가 빨간색(E)으로 나온다. Reset이나 clr을 한 번 넣어 주면 정상적으로 동작한다. VHDL에서는 내부 신호 초기값을 '0'으로 줬다.
- 출력 포트는 읽을 수 없어서 강의자료처럼 q_internal 신호를 따로 두고 q, nq에 연결했다.
- 강의자료에서는 `clk'event AND clk='1'`을 썼는데, 과제 조건에 맞춰 `rising_edge(clk)`를 사용했다.
- 문제 2, 4는 NAND로 구성하라고 되어 있어서 NOT이 필요한 곳에는 두 입력을 묶은 NAND를 썼다.
- 문제 5 게이트 회로에서 set과 clr을 그냥 넣으면 둘 다 1일 때 Q와 NQ가 둘 다 1이 되어 버린다. nPRE = NAND(set, NOT clr)로 바꿔서 VHDL의 IF-ELSIF 순서처럼 Clear가 먼저 되게 했다.

제출 파일

폴더	VHDL	Logisim
P1_rs_latch	rs_latch.vhd	rs_latch.circ
P2_d_latch	d_latch.vhd	d_latch.circ
P3_d_flipflop	d_flipflop.vhd	d_flipflop.circ
P4_jk_flipflop	jk_flipflop.vhd	jk_flipflop.circ
P5_d_flipflop_sc	d_flipflop_sc.vhd	d_flipflop_sc.circ
testbench	tb_*.vhd (5개)	-